

Unit No 3 : Level of Testing

Strategic Approach to Software Testing

A **strategic approach to software testing** is a systematic method used to plan, design, and execute tests throughout the software development lifecycle to ensure software quality.

The strategy ensures that testing activities are organized, efficient, and capable of detecting defects early.

Key Components of a Testing Strategy

1. **Test Planning**
 - Defines scope, objectives, resources, schedule, and tools.
 - Identifies testing techniques and responsibilities.
2. **Test Design**
 - Creation of **test cases, test scenarios, and test data**.
3. **Test Execution**
 - Running test cases and identifying defects.
4. **Defect Tracking**
 - Recording and managing bugs using tools like **JIRA or Bugzilla**.
5. **Test Reporting**
 - Evaluating test results and generating reports.

Levels of Testing in Strategy

Software Testing is essential to verify an application is working properly or not. To achieve this, testing can be done through various stages of development from each component to all system. Those "stages", called **Levels of Testing**, help to make sure all components of system are working properly.

Key Levels of Testing Strategy

- **Unit Testing**: The lowest level, focusing on testing individual units or components (e.g., methods, classes) in isolation to ensure they function correctly.
- **Integration Testing**: Tests the interaction between integrated units or modules to detect interface defects.
- **System Testing**: Validates the complete, integrated application as a whole to ensure it meets specified requirements.
- **Acceptance Testing**: Conducted by end-users or clients to confirm the system is ready for production and meets business needs

Importance of Strategic Testing

- Improves **software quality**
- Detects defects early
- Reduces **development cost**
- Ensures **system reliability**
- Enhances **customer satisfaction**

A. Unit Testing

Definition

Unit Testing is the process of testing the **smallest functional units of a program**, usually individual functions or modules, to verify that they work correctly.

It is usually performed by **developers during the coding phase**.

Objectives

- Verify correctness of individual modules
- Detect errors early in development
- Simplify debugging
- Improve code reliability

Characteristics

- Focuses on **smallest components**
- Usually automated
- Uses **test drivers and stubs**

Example

In a **banking system**, the following modules may be tested individually:

- Deposit function
- Withdrawal function
- Interest calculation

Advantages

- Early bug detection
- Easier debugging
- Improves code quality

Tools Used

- **JUnit**
- **NUnit**

- PyTest
-

B. Integration Testing

Definition

Integration Testing verifies that **different modules of a software system work correctly when combined together**. It is performed **after unit testing**. It's important because even if individual parts work perfectly, they might face issues when interacting with one another.

Integration testing ensures that data flows correctly between modules and that they communicate seamlessly. It helps catch problems that might arise when different parts of the system interact.

Objectives

- Identify interface errors
- Ensure proper communication between modules
- Validate data flow between components

Types of Integration Testing

1. Incremental Integration Testing

Modules are integrated **step by step** and tested gradually. **Incremental integration testing** is a software testing approach where individual software modules, which have already passed unit testing, are systematically integrated one by one or in small groups and tested after each step. The primary goal is to verify the interactions and data flow between components and identify defects early in the development lifecycle, making them easier and less costly to fix.

2. Non-Incremental Integration Testing

All modules are integrated **at once**, then tested. Non-incremental integration testing, also known as the "**Big Bang**" **approach**, is a software testing strategy where all or most developed modules are combined at once and the entire system is tested as a single entity. This method is contrasted with incremental testing, where modules are integrated and tested step-by-step.

Characteristics

- **Integration Approach:** All components are integrated simultaneously after individual unit testing is complete.

- **Planning:** Requires simpler planning as the entire testing process is done in one go.
- **Resource Efficiency:** Can be more efficient in terms of initial setup and execution time compared to planning multiple incremental steps.
- **Issue Detection:** Defects are often detected late in the development process, as testing only begins after the entire system is assembled.

When to Use Non-Incremental Testing

Non-incremental integration testing is typically used in specific circumstances:

- **Small Systems:** It is practical for small projects with a limited number of modules where the complexity of incremental testing is unnecessary.
- **Tight Coupling:** It can be necessary if components are highly interdependent and cannot operate independently of one another.
- **Strict Deadlines:** It may be used when there are tight project deadlines, although this approach carries significant risk.

Advantages and Disadvantages

Aspect	Advantages	Disadvantages
Simplicity	Simple planning and execution in one go.	High complexity for large systems, making it hard to manage.
Efficiency	Can be time-saving as the process can be automated/optimized once set up.	Can be time-consuming due to the difficulty in debugging.
Issue Isolation	Offers comprehensive, end-to-end testing of the entire system.	Extremely difficult to isolate and locate the root cause of issues, as many potential sources exist.
Risk	Allows for early detection of major, system-wide problems.	Higher risk of failure due to late error detection and potential issues in crucial modules.

3. Incremental Testing Strategies

1. Top-Down Integration Testing

Testing begins with **high-level modules** and gradually integrates lower modules. Testing begins with the highest-level control modules and moves downward to lower-level modules. [Stubs](#) (dummy programs that simulate the functionality of missing lower-level components) are used as temporary substitutes for modules that are not yet developed.

Features

- Uses **stubs**
- Tests main system structure first

Example

Testing an **online shopping system**

Order Module → Payment Module → Database Module

2. Bottom-Up Integration Testing

Testing begins with **low-level modules** and gradually moves upward. Testing starts with the lowest-level modules and progressively integrates them with higher-level ones. [Drivers](#) (dummy programs that simulate the control of missing upper-level components) are used as substitutes for the absent upper layers.

Features

- Uses **drivers**
- Low-level components tested first

Example

Database Module → Payment Module → Order Module

3. Bi-Directional Integration Testing

This method combines **Top-Down and Bottom-Up approaches**.

Advantages

- Faster testing
 - Balanced module verification
-

4. Non-Incremental Integration (Big Bang Testing)

All modules are integrated simultaneously and tested.

Advantages

- Simple implementation

Disadvantages

- Difficult to locate errors
- Debugging becomes complex

C. System testing

System Testing is when you test the software as a system. This stage checks whether the entire system functions as expected in a real-world environment. It includes both functional and non-functional tests to ensure that the software meets customer needs.

- Full end-to-end testing of the software.
- Verifying both functional and non-functional requirements.
- Testing the software's behavior in real-world conditions.

D. Acceptance Testing

Acceptance Testing, also known as **User Acceptance Testing (UAT)**, is the final test before releasing the software to the end-users. In this phase, the customer or end-users verify if the software meets their needs and expectations.

UAT is crucial because it verifies the software is ready for production. It's the last chance to catch any overlooked issues before deployment. If the software passes this stage, the customer gives the green light for release.

- Validating the software against business requirements.
- Ensuring the software meets customer expectations.
- Getting final approval from the customer.

Testing Principles

While performing the software testing, following **Testing Principles** must be applied by every software engineer:

1. The requirements of customers should be traceable and identified by all different tests.
2. Planning of tests that how tests will be conducted should be done long before the beginning of the test.
3. The Pareto principle can be applied to software testing- 80% of all errors identified during testing will likely be traceable to 20% of all program modules.
4. Testing should begin "in the small" and progress toward testing "in the large".
5. Exhaustive testing which simply means to test all the possible combinations of data is not possible.
6. Testing conducted should be most effective and for this purpose, an independent third party is required.

Acceptance Testing is the final level of software testing conducted to determine whether the software system meets the **business requirements and user needs** before it is released to the customer or market. It ensures that the system is ready for deployment and works according to the specified requirements.

Acceptance testing is usually performed **after system testing** and involves **customers, end users, or stakeholders**.

Objectives of Acceptance Testing

- Verify that the software meets **user requirements**.
 - Ensure the system works correctly in a **real-world environment**.
 - Confirm that the system is ready for **deployment or release**.
 - Identify any remaining defects before the final delivery.
-

Types of Acceptance Testing

1. Alpha Testing

Alpha testing is performed by **developers or internal testers** in the developer's environment.

Characteristics

- Conducted at the **developer's site**
- Performed by **QA team or developers**
- Executed in a **controlled environment**
- Helps detect bugs before releasing the product to external users

Example

A company developing an **online banking application** tests features like login, balance checking, and fund transfer internally before giving access to real users.

2. Beta Testing

Beta testing is performed by **real users or customers** in a real environment.

Characteristics

- Conducted at the **user's location**
- Performed by **end users**
- Helps gather **feedback and suggestions**
- Identifies issues not found during internal testing

Example

A new **mobile payment app** is released to a limited group of users to test its functionality before the official launch.

Process of Acceptance Testing

1. **Requirement Review**
Verify system requirements and user expectations.
 2. **Test Planning**
Define test scenarios and acceptance criteria.
 3. **Test Case Development**
Create test cases based on user requirements.
 4. **Test Execution**
Perform testing with real data and scenarios.
 5. **Result Evaluation**
Check whether the system meets acceptance criteria.
 6. **Approval or Rejection**
The client approves the system for deployment or requests modifications.
-

Advantages of Acceptance Testing

- Ensures the software meets **customer requirements**
 - Reduces risk of **system failure after deployment**
 - Improves **user satisfaction**
 - Identifies issues before final release
-

Limitations of Acceptance Testing

- May not detect all technical defects
 - Depends on user feedback and experience
 - Time-consuming if many users are involved
-

Example Scenario

Consider an **online shopping website**:

Acceptance testing verifies:

- Users can register and log in
- Products can be added to the cart
- Payments are processed successfully
- Order confirmation is generated

If all these functions work correctly, the system is **accepted and deployed**.

Acceptance testing is the **final stage of testing** that ensures the software is ready for release and meets the expectations of the customer or end users. It mainly includes **Alpha Testing and Beta Testing**, which help verify the system internally and externally before deployment.

Web Application Testing

Web applications require special testing due to **multiple users, browsers, and networks**. The web application testing involves detection of bugs and to ensure that quality of the websites is maintained. In order to efficiently plan, and conduct web application testing, the testers should be aware of its complete architecture, and all the critical features.

The web application testing is an integral part of complete verification of the software. It includes the validation of all the web features, parameters, browser compatibility etc. It evaluates the web errors with respect to all the errors of the total software. It is mandatory to execute the web application test cases on the different browsers, and operating systems.

Core Types of Web Application Testing

- **Functional Testing:** This ensures that all features and functions of the application work as intended according to the specified requirements. This includes:
 - **Unit Testing:** Validating individual components or code units (e.g., a specific function) in isolation.
 - **Integration Testing:** Verifying that different modules or services (e.g., the application, web server, and database) work correctly when combined.
 - **System Testing:** Testing the complete, integrated application to ensure it meets all specified requirements in a real-world environment.
 - **User Acceptance Testing (UAT):** The final stage where end-users or stakeholders verify the system meets their needs before deployment.
 - **Regression Testing:** Re-running previously conducted tests after updates or bug fixes to ensure that existing functionality hasn't been negatively impacted.
 - **Smoke Testing:** A preliminary test to ensure the core functionalities are working and the build is stable enough for further, more comprehensive testing.
- **Performance Testing:** This assesses the application's responsiveness, speed, stability, and scalability under various user loads. This includes:

- **Load Testing:** Measuring performance under expected user loads to ensure it can handle anticipated traffic.
- **Stress Testing:** Pushing the application beyond its normal operational limits to determine its breaking point and how it handles extreme conditions.
- **Scalability Testing:** Ensuring the application can scale up or down effectively in response to changing demands.
- **Security Testing:** This identifies vulnerabilities and weaknesses to protect the application and sensitive user data from potential threats and breaches. This involves:
 - **Vulnerability Scanning:** Using automated tools to scan for known security issues.
 - **Penetration Testing:** Simulating real-world attacks to find exploitable vulnerabilities.
 - **Authentication and Authorization Testing:** Verifying secure login mechanisms and proper access controls.
- **Compatibility Testing:** This ensures the web application functions and appears consistently across a variety of environments, browsers, devices, and operating systems. Key aspects include:
 - **Browser Compatibility:** Testing across browsers like Chrome, Firefox, Safari, and Edge.
 - **Operating System Compatibility:** Verifying performance on different platforms (Windows, macOS, Linux, iOS, Android).
 - **Mobile Device Compatibility:** Ensuring responsiveness and functionality on various phone and tablet screen sizes and types.
- **Usability Testing:** This focuses on the user's experience to ensure the application is intuitive, easy to navigate, and meets user expectations.
- **Accessibility Testing:** This ensures the web application is usable by individuals with disabilities and complies with relevant accessibility standards like the Web Content Accessibility Guidelines (WCAG)

Importance in Web Applications

- Ensures system can handle many users
- Prevents website crashes
- Improves customer satisfaction
- Maintains system reliability

Example

An online ticket booking system must handle **10,000 users simultaneously**.

Stability Testing

Stability testing evaluates whether a system **runs reliably for long durations without failure**.

Purpose

- Identify memory leaks
- Detect system crashes
- Ensure consistent performance

Example

A banking system must run **24/7 without downtime**.

Comparison

	Feature Scalability Testing	Stability Testing
Focus	Increasing load	Long-term reliability
Goal	System expansion	System endurance
Example	Adding more users	Running continuously for days

Acceptance Testing

Special Testing Types

1. Regression Testing

Regression testing is a software quality assurance practice that re-runs functional and non-functional tests to ensure that code changes—such as bug fixes, software updates, or new features—do not adversely affect existing, previously working functionality. It is critical for maintaining stability during rapid, iterative development cycles, preventing new bugs from introducing "collateral damage" to the system.

Key Aspects of Regression Testing

- **Purpose:** To verify that modified code hasn't broken existing features and to maintain software stability

.

- **When to Perform:** After code changes, bug fixes, software enhancements, configuration changes, or when integrating new modules.
- **Types of Testing:** Often includes a mix of unit, functional, and integration tests, typically automated for efficiency.
- **Approaches:**
 - **Full Regression:** Re-running all tests, often used for major releases.
 - **Selective Regression:** Choosing a subset of tests focused on areas impacted by code changes.
 - **Risk-Based Regression:** Prioritizing tests based on the likelihood of failure and business impact.
- **Automation:** Crucial for speed, especially in Agile and CI/CD environments where manual testing cannot keep up.
- **Comparison:** Unlike [retesting](#), which checks if a specific bug is fixed, regression testing confirms that the fix didn't create new issues elsewhere.

Common Examples

- Verifying that adding a "forgot password" feature did not break the login functionality.
- Ensuring an API update doesn't disrupt data integration with the frontend.
- Checking if modifying the database schema causes issues in existing report generation

2. GUI Testing

Graphical User Interface testing) is a type of software testing that verifies the visual elements and interactive functionality of an application to ensure they meet specifications and provide a positive user experience. This involves checking elements such as buttons, icons, menus, colors, fonts, and layouts to ensure they work correctly and appear consistently across various platforms and devices

Tests the **graphical user interface** elements such as buttons, menus, and forms. (Example: Testing whether the **login button works correctly**.)

Key Aspects of GUI Testing

- **Objective:** The primary goal is to ensure that the user interface is intuitive, visually appealing, and all interactive components function as intended from an end-user's perspective.
- **Components to Check:** Testing covers various elements, including:

- **Layout and Alignment:** Ensuring all elements are properly positioned and do not overlap when the screen is resized or viewed on different devices.
- **Visual Elements:** Verifying correct use of colors, fonts, images, and icons according to design specifications.
- **Functionality:** Confirming that interactive elements like buttons, text fields, checkboxes, and dropdown menus perform their expected actions upon user input.
- **Error Handling:** Checking that appropriate and helpful error messages are displayed when invalid data is entered or issues occur.
- **Navigation:** Ensuring that internal and external links, as well as application navigation flows, lead to the correct destinations.
- **Compatibility:** Validating that the GUI appears and functions consistently across different operating systems, browsers (cross-browser testing), and devices (mobile, desktop).

Tricentis +7

Approaches and Methods

GUI testing can be performed using both manual and automated methods, or a hybrid approach that leverages the strengths of both.

1902 Software

- **Manual Testing:** Human testers interact with the application to identify visual flaws and usability issues that automation might miss. This is especially useful for exploratory testing and subjective user experience feedback.
- **Automated Testing:** This approach uses specialized tools to run predefined scripts that mimic user interactions. It is highly effective for repetitive tasks, regression testing, and achieving wider test coverage across numerous configurations quickly and consistently.
- **Record-and-Playback:** A common automation technique where user actions are recorded and saved as test scripts for later replay. This method requires less coding knowledge but can be fragile with frequent UI changes.
- **Model-Based Testing:** This involves creating a graphical model of the system's behavior to automatically generate efficient test cases and identify potential system states